



RicozChange

AI-NATIVE CHANGE MANAGEMENT

Product PRD — everything built so far

A plain-language guide to the product: what it does, how a change flows through it, every feature shipped, and what's coming next.

VERSION

v0.2 (MVP + auth + Slack)

STATUS

Live in production

DATE

September 27, 2026

PREPARED FOR

The RicozChange team

What is RicozChange?

RicozChange is a web app for IT **change management**: the process a team follows before anything risky touches production — a database migration, a server upgrade, a config change.

Instead of approvals buried in email threads and meetings, every change is **filed in one place, scored for risk with clear reasons, approved by the right people, and checked afterwards** so the team learns from every change.

RicozChange tells you **how risky** a change is, **why, what it can break**, and gets it **approved in minutes instead of days**.

The problem it solves

The old way	The RicozChange way
Risk is a gut feeling in a meeting	Explainable risk score — every change gets a 0–100 score with line-by-line reasons
Two teams change the same system the same night and find out during the incident	The collision detector warns at filing time
CAB meetings review everything, so everything is slow	Low-risk changes auto-approve in one click; only risky ones need people
Approvers miss requests in email	Approvals arrive as Slack DMs with Approve / Reject buttons
Nobody remembers if the last change actually worked	The post-change check ("did it work?") feeds a live failure-rate dashboard

Who it's for

- **Engineers** — file changes fast, get AI-drafted rollback and test plans, see their change's risk before submitting.
- **Approvers / CAB** — decide from Slack or the approval inbox with the full "why" inline.
- **Managers** — freeze calendars, CAB meetings, imports and the failure-rate dashboard.
- **Auditors / compliance** — an append-only audit log of every action, by whom, when.

How a change flows through the product

Seven steps. Every one of them is built and running today.

1 Draft

Engineer files a change — web form, CSV import, or demo seed data.

2 Score

Risk engine scores 0–100 and shows every reason:
+70 major change, +15 business-critical system, -25 rollback plan.

3 Check

Collision detector — overlap with other changes? freeze period? bad timing?

4 Approve

Low risk → auto fast-track (one click). High risk → Slack DMs / approval inbox; CAB reviews majors.

5 Schedule

Safe-window finder suggests the best slot around freezes and neighbouring changes.

6 Implement

Owner runs the change in its window, helped by AI-drafted rollback & test plans.

7 Learn

"Did it work?" → success / partial / failed. Results feed the failure-rate dashboard... **and future risk scores** (same system failed before? the score goes up).

✓ Every step is audit-logged

Who did what, when — written once, never edited.

What the risk score looks at (all visible, no black box)

Factor	Example line item the approver sees
Change class	+70 · Base score: major change
System criticality	+15 · payments-db is business-critical (tier-0)
History of that system	+10 · 3 incidents on payments-db last quarter
Rollback plan present?	-25 · Rollback plan attached
Timing & freeze windows	+20 · Window overlaps a freeze period
Other changes in the window	+10 · Collides with change #4 on orders-api

The differentiator in one line: every competitor shows a number. RicozChange shows the number **and every reason behind it** — in the app and inside every Slack approval.

What's built — the core workflow

Feature	What it does	Where
Change requests	Create, classify (standard / normal / major / emergency), schedule a window, submit, implement, complete.	Changes page
Explainable risk score	0–100 score where every point has a reason. The product's core differentiator.	Change detail → "Why this score"
Collision detector	Warns when another change touches the same system in the same window.	Change detail + Simulator
Freeze calendar	Blocked periods (financial close, holidays) — changes landing there are rejected or flagged.	Freezes page
Safe-window finder	Suggests the best slots, avoiding freezes and neighbouring changes.	Change form
Auto fast-track	Standard changes under score 25 approve in one click — no meeting.	Any standard change
Post-change check	After the window: "did it work?" → success / partial / failed.	Change detail
CFR dashboard	Live failure-rate KPI plus 8-week trend chart.	Dashboard
CSV import	Bring your existing change backlog — kills switching cost.	Changes page

Approvals & CAB

Feature	What it does	Where
Approval inbox	All pending approvals with score, window, systems, risk reasons and one-click decisions.	Approvals page
Real Slack approvals	OAuth install → approvers get real DMs with the full "why". Approve / Reject buttons in Slack resolve the change; double-click-safe, signature-verified.	Slack workspace · Integrations page
Slack demo outbox	Renders the exact message payload — source of truth when Slack isn't connected.	Approvals page
CAB meetings	Schedule meetings, build agendas, vote, record decisions — with an outcomes donut chart.	CAB page
Any-of policy	The change is approved when any assigned approver decides; everyone else's requests auto-close. No chasing people.	Everywhere

What's built — risk tools, trust & the UI

Understanding risk (the "wow" features)

Feature	What it does	Where
Change Simulator	Drag sliders (type, window, systems...) and watch score, collisions and blast radius react live . The best demo in the product.	Simulator page
Blast-radius map	Interactive dependency graph — a change lights up everything it can take down.	Blast radius page + every change detail
AI-drafted plans	Claude drafts the rollback plan, test checklist and stakeholder comms from the change details. Human-approved, never auto-applied.	Change detail → "AI drafting"
Dashboard charts	Change volume (stacked bars by class), failure-rate trend line, class-mix donut, most-changed systems heat bars.	Dashboard

Trust & admin

Feature	What it does	Where
Audit log	Append-only trail: who did what, when — searchable, with a 14-day activity sparkline. Entries are written once and never edited.	Audit log page
Real sign-in (Clerk)	Google / email login. First sign-in with an <code>ADMIN_EMAILS</code> email auto-creates your admin account, audited as <code>user_provisioned</code> .	Every page
Roles	Engineer / approver / manager / admin — enforced on the server , not just hidden in the UI.	Everywhere
Integrations page	Live status of Slack and sign-in, plus the Slack install button.	Integrations page

The UI today

Eleven pages in the Ricoz brand — red `#d4222c` accents, deep-navy ink, a clean enterprise look. Navigation is grouped the way a real product's sidebar is:

Group	Pages
Operate	Dashboard · Changes · Approvals · CAB
Plan	Simulator · Blast radius · Freezes
Trust	Audit log · Integrations

Rules the app enforces & who can do what

Hard rules (all covered by automated tests)

- You can **never approve your own change**.
- Engineers can create and submit changes but **cannot approve anything** — the server returns 403 and logs the attempt.
- Every action lands in the audit log with a real user identity — **denials included**.
- Slack button clicks are **idempotent**; invalid signatures get 401 plus an audit entry.
- Slack down? Delivery failures are recorded and **web approvals keep working**.
- No rollback plan? The score says so — and it costs points.

Roles & permissions (enforced server-side)

Action	Engineer	Approver	Manager	Admin
View dashboards, changes, audit	✓	✓	✓	✓
Create / edit / submit own change	✓	✓	✓	✓
Approve / reject (assigned to them)	✗	✓	✓	✓
Vote in CAB	✗	✓	✓	✓
Run CAB, manage freezes, CSV import	✗	✗	✓	✓
Users, roles, settings	✗	✗	✗	✓

Current status — the honest list

Area	State
Core workflow + risk engine + charts	✓ Live — public demo, 53 backend tests passing
Real sign-in & roles	✓ Live — Clerk, gated demo, auto-provisioned admin
Slack approvals	✓ Live — OAuth install, DMs with "why", signed callbacks
Deploy	✓ Live — Render auto-deploys every push to <code>main</code>
Email-to-change	✗ Not built — first item of Phase 2
CI pipeline & error tracking	✗ Not built — Phase 2, week 4

The tech, in plain words

Piece	What we use	Why
UI	React + TypeScript + Tailwind — 11 pages	Fast to build, easy to hire for, looks like a real product.
API	FastAPI (Python), 53 automated tests	The brain: scoring, workflow, permissions.
Database	SQLite locally → managed Postgres on Render	Free to start, production-grade when it matters.
Sign-in	Clerk (Google / email)	Don't build auth yourself; SSO-ready later.
Chat	Slack app — OAuth + signed callbacks	Approvals where approvers already are.
AI	Claude API (optional)	Drafts plans; falls back to templates without a key.
Hosting	Render, auto-deploy on push to <code>main</code>	Zero-ops demo your manager can open anywhere.

Where things live in the repo

Path	What's inside
<code>backend/ricozchange/risk_engine.py</code>	Scoring, collisions, freeze handling, safe-window finder
<code>backend/ricozchange/services.py</code>	Workflow state machine, approvals, CAB, seeding, CSV import
<code>backend/ricozchange/models.py</code>	The 15 database tables (Change, System, Approval, CAB, Freeze, Audit...)
<code>backend/ricozchange/auth.py</code>	Clerk JWT verification + demo-mode fallback
<code>backend/ricozchange/slack_app.py</code> · <code>notifications.py</code>	Real Slack transport + the message payloads
<code>backend/ricozchange/main.py</code> · <code>backend/tests/</code>	All API routes · 53 end-to-end tests
<code>frontend/src/pages/</code>	One file per page (11 pages)

Run it locally (5 minutes)

1 · Backend

```
cd backend
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python -m uvicorn ricozchange.main:app --port 8000
```

2 · Frontend

```
cd frontend
npm install
npm run dev
```

3 · Open <http://localhost:5173>

The database seeds itself with demo data — including one risky pending change ready to approve.

What's next & how to talk about the product

The version ladder

Version	Theme	What gets added
v0.2 now	"Real team"	Shipped: real sign-in & roles, real Slack approvals, branded UI with charts. Remaining in Phase 2: email-to-change, CI + E2E tests, error tracking.
v0.3 ~4 weeks	"Operations"	Deploys automatically become changes (GitHub webhook), calendar sync, Microsoft Teams, DORA-style analytics.
v1.0 ~3 months	"Pilot — sellable"	Compliance pack (audit export, CAB minutes), billing & seats, SSO hardening, onboarding wizard.

The 3-minute demo script (use this with your manager)

1. **Dashboard** — failure-rate KPI, volume and trend charts.
2. Open change **#7** — walk the "Why this score" panel line by line.
3. **Approvals** — approve it with one click (or from a real Slack DM).
4. **Simulator** — drag a slider; watch score, collisions and blast radius react live.
5. Back on the change: implement → complete → post-change check → **dashboard CFR updates**.

Two-sentence pitches

To an engineer: "You file the change once; RicozChange tells you the risk, warns about collisions, drafts your rollback plan, and asks you afterwards whether it worked."

To a manager: "Every change is scored with reasons, risky ones get real approvals in Slack, low-risk ones stop wasting CAB time — and you finally get a real failure-rate number."

Glossary (for non-ITIL folks)

Term	Plain meaning		
CAB	Change Advisory Board — the group that reviews risky changes.	CFR	Change Failure Rate — % of changes that caused a problem. Lower is better; tracked live on the dashboard.
Change window	The approved time slot to do the work.	Freeze	A period when changes are blocked (financial close, holidays).
Blast radius	Everything that breaks if this change goes wrong.	Tier-0	Business-critical system (payments, checkout). Touching one costs risk points.
Rollback plan	The pre-written "undo" steps if the change fails.	Fast-track	Auto-approval for pre-approved, low-risk standard changes.